# Градиентный спуск

## 1. Теоретический минимум

В машинном обучении мы обычно хотим найти такие параметры (веса) модели $w \in \mathbb{R}^d$, при которых ошибка предсказаний минимальна. Для этого вводится **функция потерь** $L(w)$. Наша задача — решить задачу оптимизации: $w^* = \arg\min_w L(w)$.

- **Градиент** $\nabla L(w)$ — вектор частных производных: $\nabla L(w) = \left(\frac{\partial L}{\partial w_1}, \dots, \frac{\partial L}{\partial w_d}\right)^T$. Указывает направление *наискорейшего возрастания*.
- **Антиградиент** $-\nabla L(w)$ — указывает направление *наискорейшего убывания* функции.
- **Необходимое условие экстремума:** В точке оптимума $\nabla L(w) = 0$.
- **Проблема:** Градиент равен нулю также в *локальных экстремумах* и *седловых точках*. Равенство нулю не гарантирует глобального оптимума.

## 2. Аналитическое решение

Для линейной регрессии со среднеквадратичной ошибкой (MSE): $L(w) = \frac{1}{2}\|Xw - y\|^2$, где $X$ — матрица признаков, $y$ — ответы. Приравняв градиент к нулю, получаем точное решение: $w = (X^T X)^{-1} X^T y$.

**Почему это почти не используют на практике:**

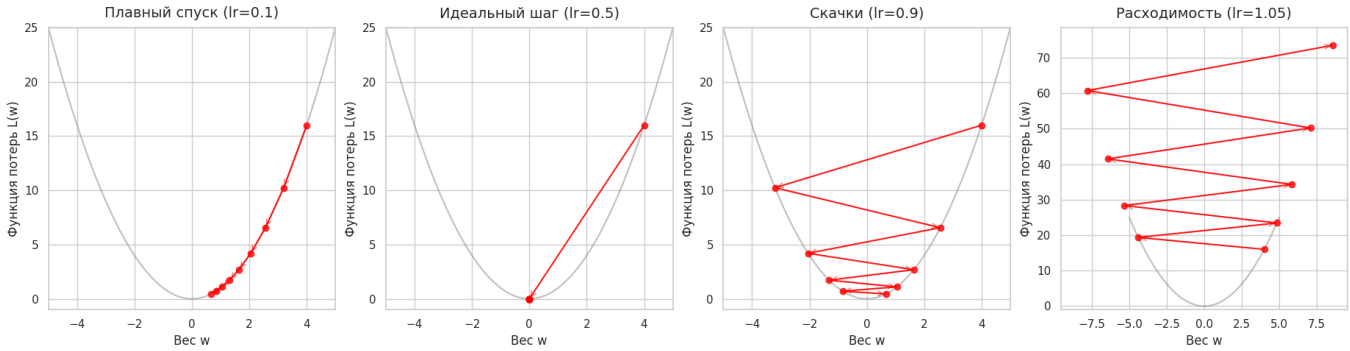
1. Считать обратную матрицу $(X^T X)^{-1}$ вычислительно долго и проблемно.GD
2. Матрица $X^T X$ может быть вырожденной (при линейно зависимых признаках).
3. Для сложных моделей аналитического решения просто не существует.

## 3. Градиентный спуск (GD)

Ищем решение итеративно! На каждом шаге идем в сторону наискорейшего уменьшения. **Формула:**

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

- $\eta$ — **learning rate** (скорость обучения или длина шага).
- **Как выбирать?** Если мал — долго идти. Если велик — скачем вокруг оптимума и можем не сойтись. На практике $\eta$ часто выбирают убывание со временем (например, $\eta_t = \frac{C}{t}$).
- **Наилучший шаг (Exact Line Search):** Аналитически ищем $\eta_t = \arg\min_\eta L(w_t - \eta \nabla L(w_t))$. Решаемо, но требует много вычислений функции потерь — это слишком дорого, **не используют на практике**




Шаги GD при разных lr

**Критерии остановки:**

1. Выполнено максимальное число итераций.
2. Норма градиента стала очень маленькой: $\|\nabla L(w_t)\| < \varepsilon$. Предполагаем, что уже в экстремуме.
3. Значение функции потерь почти перестало меняться: $|L(w_t) - L(w_{t-1})| < \varepsilon$.

**Минимальная реализация на Python:**

```
w = np.random.randn(d)
for i in range(max_iter):
    grad = get_grad(w)
    w = w - lr * grad
    if np.linalg.norm(grad) < eps:
        break
```

# 4. Стохастический спуск и батчи

Функция потерь:

$$L(w) = \frac{1}{N}\sum_{i=1}^{N} L_i(w)$$

Для одного шага GD нужно считать ошибку на *всех* $N$ объектах. Так как объектов обычно много, это слишком долго

**Стохастический градиентный спуск (SGD):** На каждом шаге берем *один случайный объект* $i$ и считаем градиент только по нему:

$$w_{t+1} = w_t - \eta \nabla L_i(w_t)$$

Вектор градиента обновляет **все веса сразу**, но основываясь лишь на одном примере. Мы жертвуем точностью направления ради большой скорости одной итерации

**Mini-batch Gradient Descent:** Берем **батч** случайных объектов размера $m$ и усредняем градиент:

$$w_{t+1} = w_t - \eta \frac{1}{m}\sum_{i=1}^{m} \nabla L_i(w_t)$$

Размер батча обычно степени двойки (32, 64, 128...) для оптимизации вычислений матриц на GPU. Дает быструю итерацию и точную оценку градиента.

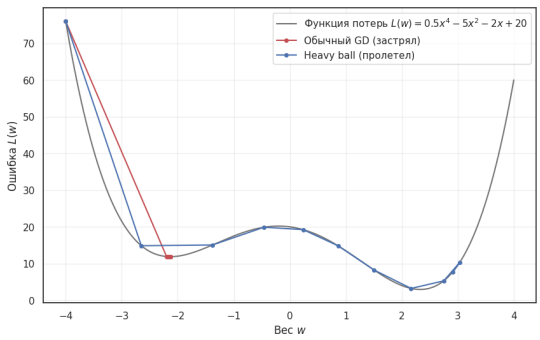
# 5. Метод тяжелого шарика (Heavy Ball)

Обычный GD часто может застревать в локальных минимумах. Позаимствуем идею из физики: добавим нашей точке «массу» и инерцию. Пусть она стремится продолжать движение в том же направлении, в котором двигалась на прошлом шаге.

**Формула:**

$$w_{t+1} = w_t - \alpha \nabla L(w_t) + \beta(w_t - w_{t-1})$$

- $\alpha$ — шаг обучения (learning rate).
- $\beta$ — коэффициент инерции (обычно $\approx 0.9$).
- $(w_t - w_{t-1})$ — вектор предыдущего сдвига.




Сравнение работы GD и Heavy ball

---

# Задачи

**Задача 1.** Дана функция потерь: $L(w_1, w_2) = 2w_1^2 + 3w_2^2 - 4w_1 + 6w_2 + 1$

1. Стартуем из $(w_1, w_2) = (0, 0)$. Найдите $\nabla L(w_1, w_2)$ и его значение в этой точке.
2. Сделайте шаг GD с $\eta = 0.1$. Где мы окажемся? Как изменилось значение $L$?
3. Найдите аналитически наилучший $\eta$ для этого первого шага

**Задача 2.** Модель предсказывает цену авто (в сотнях тысяч) от возраста: $\hat{y} = w \cdot x + b$. Функция потерь объекта: $L(w, b) = \frac{1}{2}(w \cdot x + b - y)^2$. Объект: $x = 3$ года, $y = 10$. Начальные веса: $w = 2, b = 1$.

1. Какую цену предсказывает модель сейчас? Какова ошибка? Найдите частные производные $\frac{\partial L}{\partial w}$ и $\frac{\partial L}{\partial b}$ в общем виде.
2. Подставьте $x = 3, y = 10$ и текущие веса. Сделайте один шаг SGD с $\eta = 0.1$.
3. Посчитайте предсказание с новыми весами. Что произошло и почему?

**Задача 3.** Представьте, что у нас есть два признака: $x_1$ — возраст (от 0 до 100) и $x_2$ — доход (от 0 до 1 000 000). Мы обучаем модель $y = w_1 \cdot x_1 + w_2 \cdot x_2$. Как будет выглядеть траектория обычного градиентного спуска, если мы не отнормируем данные?